DOCUMENTATION TECHNIQUE

Espace Gestionnaire

Application de gestion d'interventions, missions & frais de déplacement

Backend (Express / PostgreSQL) · Routes API · Frontend (React)
Projet **frais-deplacement-techniciens** — 10 juin 2026

Espace Gestionnaire — Documentation technique complète

Application de gestion d'interventions, missions et frais de déplacement

> Ce document explique **de zéro** toute la partie « Gestionnaire » du projet : la base de données, le backend (toutes les routes API et leurs règles de sécurité) et le frontend (chaque composant React). L'objectif est que tu comprennes le fonctionnement dans le détail **et** que tu sois capable de le refaire toi-même.

Table des matières

1. Introduction & architecture générale
2. La base de données vue par le gestionnaire
3. Authentification & sécurité par rôle
4. Le backend : panorama des routes
5. Le backend en détail (route par route)
6. Le frontend : fondations
7. Le frontend : les pages du gestionnaire
8. Trois scénarios de bout en bout
9. Les patterns à réutiliser pour coder toi-même

1. Introduction & architecture générale

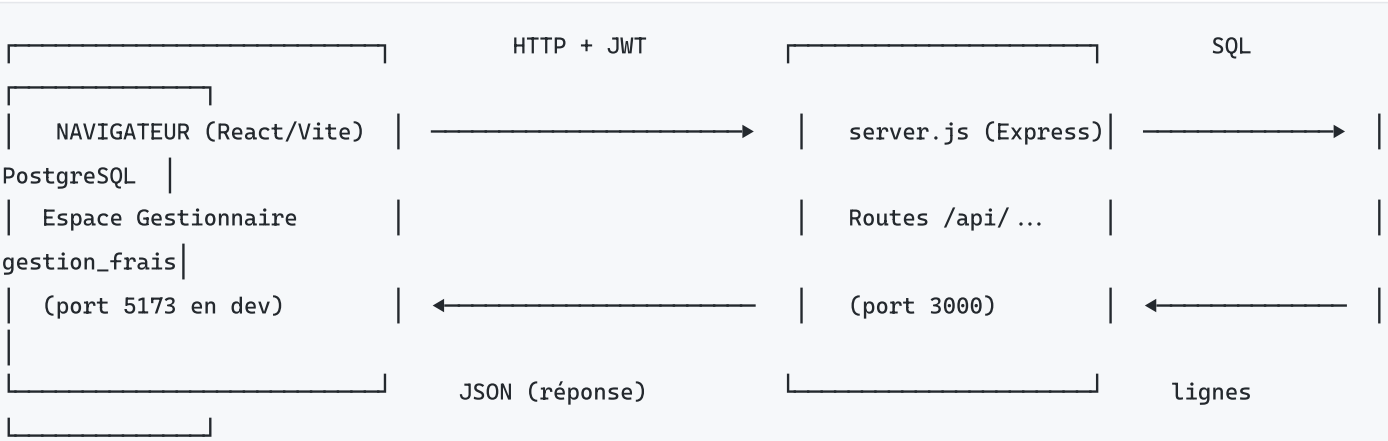
1.1 La stack technique

Couche	Technologies
Frontend	React 19, Vite, TypeScript, Tailwind CSS 4, Radix UI, react-router-dom 7, react-dnd (glisser-déposer), sonner (notifications)
Backend	Node.js, Express 5, un **seul fichier** `server.js`
Base de données	PostgreSQL (via le driver `pg`)
Sécurité	`bcrypt` (hachage des mots de passe), `jsonwebtoken` (JWT)

Point important : **le vrai backend est le fichier** `server.js` à la racine (≈ 2800 lignes). Les dossiers `backend/` (vide) et `_backendzip/` (ancienne version) **ne sont pas utilisés**.

1.2 Le schéma global

L'application est découpée en deux « espaces » indépendants : **Technicien** et **Gestionnaire**. Ce document traite l'espace **Gestionnaire**.



- Le **navigateur** affiche l'interface et n'a jamais accès direct à la base.
- Le **serveur Express** expose une API REST en `/api/ ...`, applique les **règles de sécurité** et parle à PostgreSQL.
- Chaque requête « privée » transporte un **jeton JWT** dans l'en-tête `Authorization: Bearer <token>` ; le serveur le vérifie et en déduit le rôle de l'utilisateur.

1.3 Un mot de vocabulaire

Dans cette application, **une « mission » est une** `intervention`. Les requêtes SQL renvoient d'ailleurs `i.titre AS mission`. Il n'y a pas de table `missions` séparée. Garde ça en tête : quand l'interface dit « mission », la base parle d'`intervention`.

1.4 Le cycle de vie d'une requête authentifiée

C'est LE concept central. Voici ce qui se passe quand le gestionnaire clique sur « Matériaux » :

1. Le composant React appelle `apiFetch('/api/materiaux', { token })`.
2. `apiFetch` ajoute l'en-tête `Authorization: Bearer <token>` et envoie la requête au serveur.
3. Express fait passer la requête par le **middleware** `requireRole('GESTIONNAIRE')`.
4. Le middleware décode le JWT, retrouve l'utilisateur en base, vérifie qu'il a bien le rôle `GESTIONNAIRE`. Sinon il répond `401` (non connecté) ou `403` (mauvais rôle).

5. Si tout est bon, le **handler** de la route exécute la requête SQL et renvoie du JSON.
6. `apiFetch` lit le JSON et le rend au composant, qui met à jour l'affichage.

---

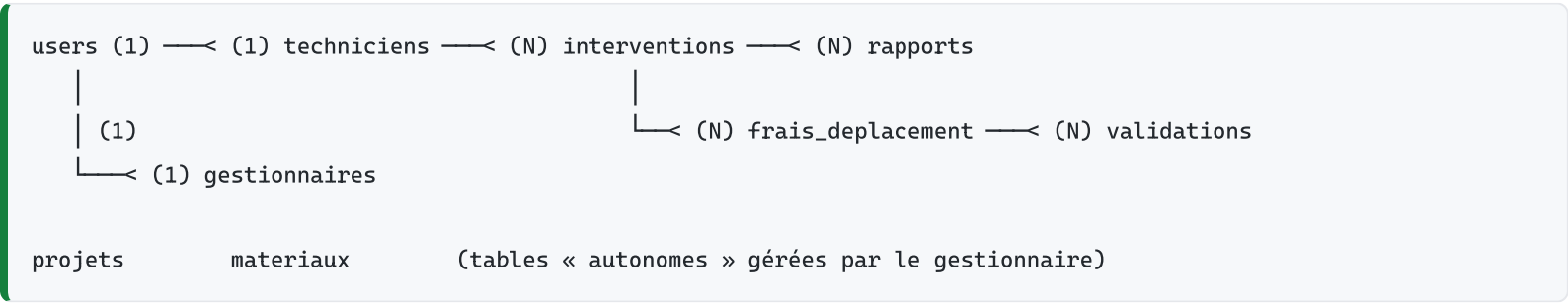
2. La base de données vue par le gestionnaire

Le gestionnaire manipule (directement ou indirectement) presque toutes les tables. Voici celles qui le concernent.

2.1 Les tables et leurs rôles

Table	Rôle	Qui écrit ?
`users`	Compte de connexion : `nom`, `prenom`, `email`, `mot_de_passe` (haché), `role`, `actif`	Gestionnaire
`techniciens`	Profil métier d'un technicien (`matricule`, `telephone`, `service`, `ville`), relié à `users`	Gestionnaire
`gestionnaires`	Profil d'un gestionnaire (`departement`), relié à `users`	Gestionnaire
`projets`	Projets/chantiers (`nom`, `client`, `localisation`, `statut`)	Gestionnaire
`materiaux`	Catalogue de matériaux (CRUD)	Gestionnaire
`interventions`	Planning : `titre`, lieux, `date_depart` (début), `date_retour` (fin), `statut`	Gestionnaire
`rapports`	Rapports d'intervention remplis par le technicien	Technicien (validés par le gestionnaire)
`frais_deplacement`	Notes de frais (liées à une intervention/mission, avec justificatif)	Technicien (validées/payées par le gestionnaire)
`validations`	Journal d'audit des décisions (validation, paiement)	Gestionnaire (automatique)

2.2 Les relations principales



- Un `user` de rôle `TECHNICIEN` possède **une** ligne `techniciens` ; un `user` de rôle `GESTIONNAIRE` possède **une** ligne `gestionnaires`. C'est le motif « table de comptes + table de profil ».
- Une `intervention` appartient à **un** technicien.
- Un `frais_deplacement` est rattaché à **une** `intervention` (= la mission) et porte un `justificatif_url`.

2.3 Définition SQL (extraits clés)

```sql
CREATE TABLE users (
  id           SERIAL PRIMARY KEY,
  nom          VARCHAR(100) NOT NULL,
  prenom       VARCHAR(100),
  email        VARCHAR(255) NOT NULL UNIQUE,
  mot_de_passe TEXT NOT NULL,          -- TOUJOURS un hash bcrypt, jamais en clair
  role         VARCHAR(50) NOT NULL DEFAULT 'TECHNICIEN',
  actif        BOOLEAN NOT NULL DEFAULT TRUE,
  created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE materiaux (
  id             SERIAL PRIMARY KEY,
  reference      VARCHAR(100) UNIQUE,
  nom            VARCHAR(255) NOT NULL,
  description    TEXT,
  categorie      VARCHAR(100),
  quantite_stock INTEGER NOT NULL DEFAULT 0,
  unite          VARCHAR(50) DEFAULT 'unité',
  prix_unitaire  NUMERIC(10,2),
  actif          BOOLEAN NOT NULL DEFAULT TRUE,
  created_at     TIMESTAMPTZ DEFAULT NOW(),
  updated_at     TIMESTAMPTZ DEFAULT NOW()
);
```

💡 **Le motif « id technique »** : chaque table a un `id SERIAL PRIMARY KEY` (entier auto-incrémenté). Les relations se font avec des clés étrangères `REFERENCES autre_table(id)`.

3. Authentification & sécurité par rôle

C'est la partie la plus importante à comprendre, car **toutes** les routes du gestionnaire en dépendent.

3.1 La connexion : `POST /login/gestionnaire`

Quand le gestionnaire se connecte, le frontend envoie son email + mot de passe. Le serveur :

1. cherche l'utilisateur par email,
2. compare le mot de passe fourni au **hash bcrypt** stocké,
3. vérifie que le rôle est bien `GESTIONNAIRE`,
4. génère un **jeton JWT** signé contenant `{ id, role }`.

```
async function handleLogin(req, res, expectedRole) {
  const { email, mot_de_passe } = req.body
  // ... recherche de l'utilisateur par email ...
  const user = result.rows[0]
  const valid = await verifyPassword(mot_de_passe, user[config.passwordColumn])
  if (!valid) return res.status(401).json({ message: 'Mot de passe incorrect' })

  // On vérifie le rôle attendu (GESTIONNAIRE pour cette route)
  if (expectedRole && normalizeRole(user.role) !== expectedRole) {
    return res.status(403).json({ message: 'Acces refuse pour ce role' })
  }

  // On fabrique un jeton valable 12h, signé avec une clé secrète
  const token = jwt.sign({ id: user.id, role: normalizeRole(user.role) }, jwtSecret, {
    expiresIn: '12h',
  })
  return res.json({ message: 'Connexion reussie', token, role: normalizeRole(user.role) })
}

// Trois portes d'entrée, la même fonction :
app.post('/login',               (req, res) => handleLogin(req, res, null))
app.post('/login/technicien',    (req, res) => handleLogin(req, res, ROLE_TECHNICIEN))
app.post('/login/gestionnaire',  (req, res) => handleLogin(req, res, ROLE_GESTIONNAIRE))
```

Le mot de passe est comparé via `bcrypt` (qui sait re-hacher et comparer sans jamais déchiffrer) :

```
async function verifyPassword(plainPassword, storedPassword) {
  if (looksLikeBcryptHash(storedPassword)) {
    return bcrypt.compare(plainPassword, storedPassword)
  }
  return plainPassword === storedPassword // tolérance pour d'anciens comptes en clair
}
```

3.2 Le jeton JWT, en deux mots

Un JWT est une chaîne signée de la forme `entête.charge_utile.signature`. La **charge utile** ici contient `{ id: 42, role: "GESTIONNAIRE" }`. La **signature** est calculée avec une clé secrète (`JWT_SECRET` dans le `.env`). Personne ne peut fabriquer un

faux jeton sans connaître cette clé, et le serveur peut vérifier l'authenticité **sans interroger la base** (sauf qu'ici on relit l'utilisateur pour être sûr qu'il existe encore).

3.3 Le middleware `requireRole(role)` — le videur à l'entrée

Chaque route privée est « protégée » par ce middleware. C'est une fonction qui **renvoie** une autre fonction (un *middleware* Express : `(req, res, next)`).

```
async function authFromRequest(req) {
  const header = req.headers.authorization
  if (!header || !header.startsWith('Bearer ')) throw new Error('Token manquant')

  const token = header.slice('Bearer '.length)
  const payload = jwt.verify(token, jwtSecret)         // vérifie la signature + l'expiration

  const user = await fetchUserById(payload.id)         // l'utilisateur existe-t-il encore ?
  if (!user) throw new Error('Utilisateur introuvable')

  return { user, role: normalizeRole(payload.role || user.role) }
}

function requireRole(role) {
  return async (req, res, next) => {
    try {
      const auth = await authFromRequest(req)
      if (role && auth.role !== role) {
        return res.status(403).json({ error: 'Acces refuse pour ce role' }) // mauvais rôle
      }
      req.auth = auth      // on attache l'utilisateur à la requête pour le handler
      next()               // feu vert : on passe au handler de la route
    } catch (error) {
      return res.status(401).json({ error: error.message || 'Non autorise' }) // pas/plus connecté
    }
  }
}
```

À retenir :

- `requireRole(ROLE_GESTIONNAIRE)` → réservé au gestionnaire.
- `requireRole(ROLE_TECHNICIEN)` → réservé au technicien.
- `requireRole(null)` → n'importe quel utilisateur **connecté** (rôle indifférent).
- Après le middleware, le handler dispose de `req.auth.user` (l'utilisateur courant) et `req.auth.role`.

La différence **401 vs 403** est essentielle :

- **401 Unauthorized** : « je ne sais pas qui tu es » (jeton absent, invalide ou expiré).
- **403 Forbidden** : « je sais qui tu es, mais tu n'as pas le droit » (mauvais rôle).

3.4 Côté frontend : stocker et envoyer le jeton

Le jeton reçu à la connexion est rangé dans le `sessionStorage` du navigateur, puis renvoyé à chaque appel.

```ts
// managerSession.ts — un petit module qui isole la gestion du jeton
const MANAGER_TOKEN_KEY = 'managerToken'

export function setManagerSession(token: string) { sessionStorage.setItem(MANAGER_TOKEN_KEY, token) }
export function clearManagerSession()             { sessionStorage.removeItem(MANAGER_TOKEN_KEY) }
export function getManagerToken(): string | null  { return sessionStorage.getItem(MANAGER_TOKEN_KEY) }
export function isManagerAuthenticated(): boolean  { return Boolean(getManagerToken()) }
```

Le `sessionStorage` se vide à la fermeture de l'onglet : c'est volontaire (sécurité). Pour une session persistante, on utiliserait `localStorage`.

3.5 La garde de route `RequireManagerAuth`

Côté React, on empêche d'afficher les pages du gestionnaire si aucun jeton n'est présent. C'est le rôle de ce minuscule composant, branché dans le routeur (voir §6).

```tsx
import { Navigate, Outlet } from 'react-router-dom'
import { isManagerAuthenticated } from './managerSession'

export default function RequireManagerAuth() {
  if (!isManagerAuthenticated()) {
    return <Navigate to="/gestionnaire/login" replace />  // redirection si non connecté
  }
  return <Outlet />   // sinon, on affiche la page demandée
}
```

⚠️ Cette garde n'est qu'une **commodité d'affichage**. La vraie sécurité est côté serveur (`requireRole`). On ne fait jamais confiance au frontend pour la sécurité.

4. Le backend : panorama des routes

Toutes les routes ci-dessous sont dans `server.js`. La colonne « Accès » indique le middleware appliqué.

Méthode & route	Accès	Rôle fonctionnel
`POST /login/gestionnaire`	public	Connexion gestionnaire
`GET /api/users`	GESTIONNAIRE	Liste des comptes (filtres `role`, `actif`)
`GET /api/users/:id`	GESTIONNAIRE	Détail d'un compte
`POST /api/users`	GESTIONNAIRE	Créer un compte (technicien ou gestionnaire)
`PUT /api/users/:id`	GESTIONNAIRE	Modifier un compte
`DELETE /api/users/:id`	GESTIONNAIRE	Désactiver un compte (soft delete)
`POST /api/users/:id/reset-password`	GESTIONNAIRE	Réinitialiser le mot de passe
`GET/POST/PUT/DELETE /api/techniciens …`	GESTIONNAIRE	CRUD techniciens (variante)
`GET/POST/PUT/DELETE /api/gestionnaires …`	GESTIONNAIRE	CRUD gestionnaires
`GET /api/projets`	connecté	Liste des projets (partagée)
`POST/PUT/DELETE /api/projets …`	GESTIONNAIRE	CRUD projets
`GET/POST/PUT/DELETE /api/materiaux …`	GESTIONNAIRE	CRUD catalogue matériaux
`GET /api/interventions`	GESTIONNAIRE	Planning (filtre par semaine)
`POST/PUT/DELETE /api/interventions …`	GESTIONNAIRE	CRUD interventions
`GET /api/rapports/stats`	GESTIONNAIRE	Compteurs de rapports
`GET /api/rapports`	connecté	Liste des rapports
`PUT /api/rapports/:id/valider`	GESTIONNAIRE	Valider/rejeter un rapport
`GET /api/manager/dashboard`	GESTIONNAIRE	Statistiques du tableau de bord
`GET /api/manager/expenses`	GESTIONNAIRE	Liste des notes de frais (filtre statut)
`PATCH /api/manager/expenses/:id/validate`	GESTIONNAIRE	Valider une note de frais
`PATCH /api/manager/expenses/pay`	GESTIONNAIRE	Payer (rembourser) plusieurs frais

4.1 Les briques communes du backend

Avant de détailler les routes, voici les fonctions utilitaires réutilisées partout.

```
// Convertit "42" en 42, et renvoie null si ce n'est pas un entier valide.
function parseInteger(value) {
  const n = Number.parseInt(String(value), 10)
  return Number.isFinite(n) ? n : null
}

// Transforme une ligne SQL (snake_case) en objet JSON (camelCase) pour le front.
function formatMaterialRow(row) {
  return {
    id: row.id,
    reference: row.reference || null,
    nom: row.nom,
    quantiteStock: Number(row.quantite_stock ?? 0),
    prixUnitaire: row.prix_unitaire !== null ? Number(row.prix_unitaire) : null,
    actif: row.actif === true,
    // ...
  }
}

// Exécute plusieurs requêtes dans UNE transaction (tout réussit, ou tout est annulé).
async function runTransaction(work) {
  const client = await pool.connect()
  try {
    await client.query('BEGIN')
    const result = await work(client)
    await client.query('COMMIT')
    return result
  } catch (error) {
    await client.query('ROLLBACK')   // en cas d'erreur, on annule TOUT
    throw error
  } finally {
    client.release()
  }
}
```

Trois idées fondamentales s'y cachent :

1. **Requêtes paramétrées** (`$1` , `$2` , ...) : on ne colle JAMAIS une valeur utilisateur directement dans le SQL → protection contre l'**injection SQL**.
2. **Formatage des lignes** : le SQL parle `snake_case` , le frontend préfère `camelCase` ; les fonctions `formatXxxRow` font le pont.
3. **Transactions** : quand une opération touche plusieurs tables (ex. créer un `user` + son profil `techniciens`), on les regroupe pour garantir la cohérence.

5. Le backend en détail (route par route)

5.1 Gestion des comptes — `users`

Lister les comptes — `GET /api/users`

Cette route alimente la page « Ressources » et la liste des techniciens du planning. Elle accepte deux filtres en *query string* : `?role=TECHNICIEN` et `?actif=true`.

```
app.get('/api/users', requireRole(ROLE_GESTIONNAIRE), async (req, res) => {
  const filters = []
  const params = []

  // On construit dynamiquement le WHERE en fonction des filtres présents.
  const role = parseRoleFilter(req.query.role)
  if (role) { params.push(role); filters.push(`u.role = $${params.length}`) }

  const actif = parseActiveFilter(req.query.actif ?? req.query.statut)
  if (actif !== null) { params.push(actif); filters.push(`u.actif = $${params.length}`) }

  const whereClause = filters.length ? `WHERE ${filters.join(' AND ')}` : ''

  const result = await pool.query(`
    SELECT u.id, u.nom, u.prenom, u.email, u.role, u.actif, u.created_at,
           t.id AS technicien_id, t.matricule, t.telephone, t.service, t.ville,
           g.id AS gestionnaire_id, g.departement
    FROM users u
    LEFT JOIN techniciens t    ON t.user_id = u.id
    LEFT JOIN gestionnaires g  ON g.user_id = u.id
    ${whereClause}
    ORDER BY u.created_at DESC, u.id DESC
  `, params)

  return res.json({ success: true, count: result.rows.length, users: result.rows.map(formatUserRow) })
})
```

Deux techniques à retenir :

- **WHERE dynamique** : on n'ajoute une condition que si le filtre est fourni, en synchronisant `params` et le numéro `$N`.
- `LEFT JOIN` : on rapatrie en une requête le compte ET son profil (technicien ou gestionnaire), même si le profil n'existe pas.

Créer un compte — `POST /api/users`

Création atomique : on insère dans `users` **puis** dans `techniciens` (ou `gestionnaires`) selon le rôle, le tout dans une transaction. Le mot de passe est **haché** avant insertion.

```
app.post('/api/users', requireRole(ROLE_GESTIONNAIRE), async (req, res) => {
  const { nom, email } = req.body
  const password = req.body.password ?? req.body.mot_de_passe
  const role = parseRoleFilter(req.body.role) || ROLE_TECHNICIEN
  if (!nom || !email || !password) {
    return res.status(400).json({ error: 'nom, email et password sont obligatoires' })
  }

  // Email déjà pris ?
  const existing = await pool.query('SELECT id FROM users WHERE email = $1 LIMIT 1', [email])
  if (existing.rows.length) return res.status(409).json({ error: 'Cet email existe déjà' })

  const hashedPassword = await bcrypt.hash(password, 10)   // 10 = coût du hachage

  const createdUser = await runTransaction(async (client) => {
    const userInsert = await client.query(
      `INSERT INTO users (nom, prenom, email, mot_de_passe, role, actif)
       VALUES ($1,$2,$3,$4,$5,$6) RETURNING id`,
      [nom, prenom, email, hashedPassword, role, actif],
    )
    const userId = userInsert.rows[0].id
    if (role === ROLE_TECHNICIEN)   await syncTechnicianProfile(client, userId, { /* matricule … */ })
    else if (role === ROLE_GESTIONNAIRE) await syncManagerProfile(client, userId, { /* departement */ })
    return { id: userId }
  })

  const user = await fetchUserDetailsById(createdUser.id)
  return res.status(201).json({ success: true, user: formatUserRow(user) })
})
```

Codes de statut utilisés : `400` (champ manquant), `409` (conflit : email déjà pris), `201` (créé).

Modifier un compte — `PUT /api/users/:id` (avec une règle de sécurité)

La modification suit le même schéma, **mais** elle applique une règle métier importante : **un gestionnaire ne peut pas changer le mot de passe d'un technicien**.

```
// Règle : le gestionnaire NE PEUT PAS changer le mot de passe d'un technicien.
if (password && normalizeRole(existingUser.role) === ROLE_TECHNICIEN) {
  return res.status(403).json({
    error: "Le gestionnaire ne peut pas changer le mot de passe d'un technicien. Utilisez la réinitialisation.",
  })
}
```

Supprimer un compte — `DELETE /api/users/:id` (soft delete)

On ne **supprime** pas réellement la ligne (cela casserait les interventions/frais liés). On la **désactive** : `actif = false`. C'est un *soft delete*.

```
app.delete('/api/users/:id', requireRole(ROLE_GESTIONNAIRE), async (req, res) => {
  const result = await pool.query('UPDATE users SET actif = false WHERE id = $1 RETURNING id', [userId])
  if (!result.rows.length) return res.status(404).json({ error: 'Utilisateur introuvable' })
  return res.json({ success: true, message: 'Utilisateur désactivé' })
})
```

Réinitialiser le mot de passe — `POST /api/users/:id/reset-password`

Le gestionnaire ne **choisit** pas le nouveau mot de passe : le serveur en **génère** un temporaire, le hache, et le renvoie **une seule fois** pour que le gestionnaire le communique au technicien.

```
app.post('/api/users/:id/reset-password', requireRole(ROLE_GESTIONNAIRE), async (req, res) => {
  const temporaryPassword = generateTemporaryPassword()        // ex: "Temp-9f3a1c2b"
  const hashed = await bcrypt.hash(temporaryPassword, 10)
  await pool.query('UPDATE users SET mot_de_passe = $1 WHERE id = $2', [hashed, userId])
  return res.json({ success: true, temporaryPassword })        // affiché une fois, jamais re-stocké en clair
})
```

🔐 **Pourquoi cette séparation ?** Le gestionnaire administre les comptes mais ne doit pas connaître les mots de passe permanents des techniciens. La réinitialisation donne un accès temporaire ; le technicien le change ensuite lui-même (route `PUT /api/technicien/password`).

5.2 Le catalogue de matériaux — `materiaux` (CRUD complet)

C'est l'exemple **type** d'un module CRUD. Les 5 routes sont toutes en `requireRole(GESTIONNAIRE)`, donc un technicien reçoit automatiquement `403`.

Lister + rechercher — `GET /api/materiaux?search=…&actif=…`

```
app.get('/api/materiaux', requireRole(ROLE_GESTIONNAIRE), async (req, res) => {
  const filters = []; const params = []

  const search = (req.query.search || '').trim()
  if (search) {
    params.push(`%${search}%`)
    filters.push(`(m.nom ILIKE $${params.length} OR m.reference ILIKE $${params.length} OR m.categorie ILIKE
$${params.length})`)
  }
  const actif = parseActiveFilter(req.query.actif)
  if (actif !== null) { params.push(actif); filters.push(`m.actif = $${params.length}`) }

  const whereClause = filters.length ? `WHERE ${filters.join(' AND ')}` : ''
  const result = await pool.query(`SELECT m.* FROM materiaux m ${whereClause} ORDER BY m.nom ASC`, params)
  return res.json({ success: true, count: result.rows.length, materiaux: result.rows.map(formatMaterialRow) })
})
```

`ILIKE` = recherche insensible à la casse ; `%term%` = « contient ». La recherche couvre le nom, la référence et la catégorie.

Créer — `POST /api/materiaux`

```
app.post('/api/materiaux', requireRole(ROLE_GESTIONNAIRE), async (req, res) => {
  const nom = (req.body.nom || '').trim()
  if (!nom) return res.status(400).json({ error: 'Le nom du matériau est obligatoire' })

  // référence unique : on vérifie avant d'insérer
  if (reference) {
    const dup = await pool.query('SELECT id FROM materiaux WHERE reference = $1 LIMIT 1', [reference])
    if (dup.rows.length) return res.status(409).json({ error: 'Cette référence existe déjà' })
  }

  const result = await pool.query(
    `INSERT INTO materiaux (reference, nom, description, categorie, quantite_stock, unite, prix_unitaire, actif)
     VALUES ($1,$2,$3,$4,$5,$6,$7,$8) RETURNING *`,
    [reference, nom, description, categorie, quantiteStock, unite, prixUnitaire, actif],
  )
  return res.status(201).json({ success: true, materiau: formatMaterialRow(result.rows[0]) })
})
```

Modifier / Supprimer — `PUT /:id` et `DELETE /:id`

`PUT` relit d'abord la ligne existante, fusionne les nouvelles valeurs (chaque champ absent garde sa valeur actuelle), met à jour `updated_at = NOW()`. `DELETE` supprime réellement la ligne :

```
app.delete('/api/materiaux/:id', requireRole(ROLE_GESTIONNAIRE), async (req, res) => {
  const result = await pool.query('DELETE FROM materiaux WHERE id = $1 RETURNING id', [id])
  if (!result.rows.length) return res.status(404).json({ error: 'Matériau introuvable' })
  return res.json({ success: true, message: 'Matériau supprimé' })
})
```

💡 **CRUD = Create, Read, Update, Delete.** Chaque module du gestionnaire (users, projets, materiaux, interventions) suit exactement ce squelette : `GET` (liste), `GET /:id` (détail), `POST` (créer), `PUT /:id` (modifier), `DELETE /:id` (supprimer). Une fois que tu en as compris un, tu les comprends tous.

5.3 Le planning — `interventions` (avec gestion multi-jours)

Lister une semaine — `GET /api/interventions?semaine=YYYY-MM-DD`

Le planning affiche une semaine. Comme une intervention peut **durer plusieurs jours**, on ne sélectionne pas seulement celles qui *commencent* dans la semaine, mais toutes celles qui la **chevauchent**.

```
// Une intervention chevauche [début, fin] si :
//   date_depart ≤ fin  ET  date_fin (= date_retour) ≥ début
params.push(startDate.toISOString()); const startParam = params.length
params.push(endDate.toISOString());   const endParam   = params.length
filters.push(
  `i.date_depart < ($${endParam}::timestamptz + interval '1 day')
   AND COALESCE(i.date_retour, i.date_depart) ≥ $${startParam}`,
)
```

`COALESCE(i.date_retour, i.date_depart)` = « prends `date_retour`, ou à défaut `date_depart` » : cela gère les anciennes interventions qui n'avaient pas de date de fin. La requête joint aussi le technicien (nom, prénom, email) pour l'affichage.

Créer une intervention — `POST /api/interventions`

```
const dateDepart = new Date(req.body.date_depart ?? req.body.date_debut)
let dateRetour = dateDepart                        // par défaut : 1 seule journée
if (req.body.date_fin) {
  const parsed = new Date(req.body.date_fin)
  if (parsed < dateDepart) {
    return res.status(400).json({ error: 'La date de fin doit être postérieure ou égale au début' })
  }
  dateRetour = parsed
}
await pool.query(
  `INSERT INTO interventions (technicien_id, titre, description, lieu_depart, lieu_arrivee, date_depart,
date_retour, statut)
   VALUES ($1,$2,$3,$4,$5,$6,$7,$8) RETURNING id`,
  [techId, titre, description, lieuDepart, lieuArrivee, dateDepart.toISOString(), dateRetour.toISOString(),
statut],
)
```

`PUT /api/interventions/:id` met à jour les mêmes champs via `COALESCE($n, colonne)` (ne change que ce qui est fourni). C'est ce que le **glisser-déposer** appelle pour réaffecter une intervention à un autre technicien ou un autre jour.

5.4 La validation des rapports — `PUT /api/rapports/:id/valider`

Le gestionnaire valide ou rejette un rapport d'intervention. La décision (`VALIDE` / `REJETE`), le validateur (`req.auth.user.id`) et la date sont enregistrés.

```
app.put('/api/rapports/:id/valider', requireRole(ROLE_GESTIONNAIRE), async (req, res) => {
  const decision = parseReportDecision(req.body.decision)        // 'VALIDE' ou 'REJETE'
  if (!decision) return res.status(400).json({ error: 'Décision invalide' })

  await runTransaction(async (client) => {
    await client.query(
      `UPDATE rapports
       SET statut = $1, validate_par = $2, date_validation = NOW(), commentaire_validation = $3
       WHERE id = $4 RETURNING id`,
      [decision, req.auth.user.id, commentaire, reportId],
    )
  })
  // ... puis on relit le rapport enrichi (technicien, intervention, validateur) et on le renvoie
})
```

`GET /api/rapports/stats` renvoie simplement trois compteurs (`enAttente` , `valides` , `rejetes`) calculés par trois `COUNT(*)` .

5.5 Le tableau de bord — `GET /api/manager/dashboard`

Cette route agrège **tout** ce qu'affiche la page d'accueil : compteurs + dernières activités. Astuce de performance : toutes les requêtes sont lancées **en parallèle** avec `Promise.all`.

```
const [techCount, intervCount, expenseCount, pendingCount, validatedCount, recentInterv, recentExp] =
  await Promise.all([
    pool.query(`SELECT COUNT(*)::int AS count FROM techniciens t JOIN users u ON u.id=t.user_id WHERE
u.actif=true`),
    pool.query('SELECT COUNT(*)::int AS count FROM interventions'),
    pool.query('SELECT COUNT(*)::int AS count FROM frais_deplacement'),
    pool.query('SELECT COUNT(*)::int AS count FROM frais_deplacement WHERE statut_validation = $1',
['EN_ATTENTE']),
    pool.query('SELECT COUNT(*)::int AS count FROM frais_deplacement WHERE statut_validation = $1', ['VALIDEE']),
    pool.query(/* 5 dernières interventions + nom technicien */),
    pool.query(/* 5 dernières notes de frais + mission + technicien */),
  ])
return res.json({ manager, stats: { /* ... */ }, recentInterventions, recentExpenses })
```

5.6 Le circuit des notes de frais — `expenses`

C'est un **workflow** en trois états :

```
EN_ATTENTE ──(gestionnaire valide)──▶ VALIDEE ──(gestionnaire paie)──▶ PAYEE
```

Lister — `GET /api/manager/expenses?status=EN_ATTENTE`

Renvoie les frais filtrés par statut, enrichis de la mission (`i.titre AS mission`), du technicien et de la dernière décision. C'est la même requête qui sert à la page « Validation » (`status=EN_ATTENTE`) et à la page « Paiements » (`status=VALIDEE`).

Valider — `PATCH /api/manager/expenses/:id/validate`

```
const updatedId = await runTransaction(async (client) => {
  // On ne valide QUE si le frais est encore "EN_ATTENTE" (évite la double validation)
  const result = await client.query(
    `UPDATE frais_deplacement SET statut_validation = $1
     WHERE id = $2 AND statut_validation = $3 RETURNING id`,
    [EXPENSE_STATUS_VALIDATED, expenseId, EXPENSE_STATUS_PENDING],
  )
  if (!result.rows[0]) return null
  // Trace dans le journal d'audit
  await client.query(
    `INSERT INTO validations (frais_id, gestionnaire_id, decision, commentaire) VALUES ($1,$2,$3,$4)`,
    [expenseId, manager.manager_id, EXPENSE_STATUS_VALIDATED, commentaire],
  )
  return result.rows[0].id
})
if (!updatedId) return res.status(404).json({ error: 'Frais introuvable ou déjà traité' })
```

Payer en lot — `PATCH /api/manager/expenses/pay`

Le gestionnaire coche plusieurs frais validés et les rembourse en une fois. On passe un **tableau d'ids**.

```
const expenseIds = ids.map(parseInteger).filter(Boolean)
await pool.query(
  `UPDATE frais_deplacement SET statut_validation = $1 WHERE id = ANY($2::int[]) RETURNING id`,
  [EXPENSE_STATUS_PAID, expenseIds],     // ANY($2::int[]) = "id parmi cette liste"
)
// Une ligne d'audit par frais payé
await pool.query(
  `INSERT INTO validations (frais_id, gestionnaire_id, decision, commentaire)
   SELECT f.id, $1, $2, $3 FROM frais_deplacement f WHERE f.id = ANY($4::int[])`,
  [manager.manager_id, EXPENSE_STATUS_PAID, reference ? `Paiement … Reference: ${reference}` : 'Paiement
effectue', expenseIds],
)
```

`id = ANY($2::int[])` est l'équivalent SQL de « id ∈ {liste} » : pratique pour agir sur plusieurs lignes d'un coup.

6. Le frontend : fondations

Avant les pages, comprenons les 4 briques qui les font fonctionner.

6.1 Le client API — `lib/api.ts`

Toutes les requêtes passent par **une seule fonction** `apiFetch`. Elle ajoute le jeton, sérialise le corps en JSON, et transforme les erreurs HTTP en exceptions.

```
export async function apiFetch<T>(path, options = {}): Promise<T> {
  const headers = new Headers(options.headers)
  if (options.token) headers.set('Authorization', `Bearer ${options.token}`)   // ← le jeton

  let body = options.body
  if (body && typeof body === 'object' && !(body instanceof FormData)) {
    headers.set('Content-Type', 'application/json')
    body = JSON.stringify(body)
  }

  const response = await fetch(`${API_BASE_URL}${path}`, { ...options, headers, body })
  const data = /* json si content-type json, sinon texte */

  if (!response.ok) {                       // 4xx / 5xx → on lève une ApiError
    throw new ApiError(data?.error || data?.message || 'Erreur API', response.status)
  }
  return data as T
}
```

Avantage : chaque composant écrit `await apiFetch('/api/...', { token })` sans répéter la logique d'en-têtes, de JSON et d'erreurs.

6.2 Le routeur — `routes/appRoutes.tsx`

Le routeur déclare les URLs. Les pages du gestionnaire sont **regroupées** sous la garde `RequireManagerAuth` : impossible d'y accéder sans jeton.

```
export const router = createBrowserRouter([
  { path: '/', Component: Root, children: [
    { index: true, Component: LandingPage },

    { path: 'gestionnaire/login', Component: ManagerLogin },    // public
    {
      Component: RequireManagerAuth,                            // ← garde commune
      children: [
        { path: 'gestionnaire/dashboard',  Component: ManagerDashboard },
        { path: 'gestionnaire/ressources', Component: ManagerResources },
        { path: 'gestionnaire/materiaux',  Component: ManagerMaterials },
        { path: 'gestionnaire/planning',   Component: ManagerPlanning },
        { path: 'gestionnaire/validation', Component: ManagerValidation },
        { path: 'gestionnaire/paiements',  Component: ManagerPayments },
      ],
    },
  ]},
])
```

6.3 La session — `managerSession.ts`

Déjà vue au §3.4 : `setManagerSession` / `getManagerToken` / `clearManagerSession`. C'est le seul endroit qui connaît la clé `'managerToken'`.

6.4 Le gabarit visuel — `ManagerLayout.tsx`

Toutes les pages du gestionnaire sont enveloppées dans ce composant qui affiche la **barre latérale** (navigation) et le bouton de déconnexion. La page réelle est passée en `children`.

```
export default function ManagerLayout({ children }) {
  const navigation = [
    { name: 'Tableau de bord', path: '/gestionnaire/dashboard', icon: LayoutDashboard },
    { name: 'Ressources',      path: '/gestionnaire/ressources', icon: Users },
    { name: 'Matériaux',       path: '/gestionnaire/materiaux',  icon: Package },
    { name: 'Planning Hebdo',  path: '/gestionnaire/planning',   icon: Calendar },
    { name: 'Validation',      path: '/gestionnaire/validation', icon: CheckSquare },
    { name: 'Paiements',       path: '/gestionnaire/paiements',  icon: CreditCard },
  ]
  const handleLogout = () => { clearManagerSession(); navigate('/gestionnaire/login', { replace: true }) }
  return (
    <div className="flex h-screen">
      <aside> {/* logo + navigation.map( … ) + bouton Déconnexion */} </aside>
      <main className="flex-1 overflow-auto">{children}</main>
    </div>
  )
}
```

7. Le frontend : les pages du gestionnaire

Toutes les pages suivent le **même cycle de vie** : un état local (`useState`), un chargement de données au montage (`useEffect` + `apiFetch`), un rendu conditionnel (chargement / vide / liste), et des actions (créer, modifier…) qui rappellent l'API puis rechargent.

7.1 La connexion — `ManagerLogin.tsx`

```
const handleLogin = async (e) => {
  e.preventDefault()
  const data = await apiFetch<{ token: string }>('/login/gestionnaire', {
    method: 'POST',
    body: { email, mot_de_passe: password },
  })
  setManagerSession(data.token)                        // on range le jeton
  navigate('/gestionnaire/dashboard', { replace: true })   // on entre dans l'espace
}
```

7.2 Le tableau de bord — `ManagerDashboard.tsx`

Au montage, il appelle `/api/manager/dashboard` et affiche 4 cartes de statistiques + deux listes (activité récente, dernières notes de frais).

```
useEffect(() => {
  const token = getManagerToken()
  apiFetch<DashboardData>('/api/manager/dashboard', { token })
    .then(setData).catch(() => {}).finally(() => setIsLoading(false))
}, [])
```

Le motif `isLoading ? "Chargement…" : <contenu>` évite d'afficher une page vide pendant l'attente réseau.

7.3 Les ressources — `ManagerResources.tsx`

Page à **onglets** (Radix `Tabs`) : « Techniciens » et « Projets ». Chacun a son CRUD (ajout/édition/suppression via des modales `Dialog`).

```
const loadResources = async () => {
  const token = getManagerToken()
  const [techData, projData] = await Promise.all([
    apiFetch<{ users: TechnicianRow[] }>('/api/users?role=TECHNICIEN&actif=true', { token }),
    apiFetch<{ projets: ProjectRow[] }>('/api/projets', { token }),
  ])
  setTechnicians(techData.users); setProjects(projData.projets)
}
```

C'est ici qu'on applique la **règle des mots de passe** côté interface : la modale d'édition d'un technicien **ne propose plus** de champ « mot de passe ». À la place, un bouton « Réinitialiser le mot de passe » :

```
const handleResetPassword = async () => {
  const res = await apiFetch<{ temporaryPassword: string }>(
    `/api/users/${editingTech.id}/reset-password`, { method: 'POST', token },
  )
  setTemporaryPassword(res.temporaryPassword)    // affiché dans un encadré, à recopier
}
```

7.4 Les matériaux — `ManagerMaterials.tsx`

CRUD complet du catalogue : un tableau, un champ de recherche (avec *debounce*), une modale d'ajout/édition, une suppression confirmée.

```
// Recherche "anti-rafale" : on attend 250ms après la dernière frappe avant d'appeler l'API.
useEffect(() => {
  const timer = setTimeout(loadMaterials, 250)
  return () => clearTimeout(timer)
}, [search])

const handleSave = async () => {
  const payload = { reference, nom, categorie, quantite_stock, unite, prix_unitaire }
  if (editingId) await apiFetch(`/api/materiaux/${editingId}`, { method: 'PUT',  token, body: payload })
  else           await apiFetch('/api/materiaux',            { method: 'POST', token, body: payload })
  await loadMaterials()      // on recharge la liste pour refléter le changement
}
```

Le *debounce* est un réflexe à connaître : sans lui, on enverrait une requête à chaque lettre tapée.

7.5 Le planning — `ManagerPlanningDragDrop.tsx`

La page la plus riche. Elle affiche une **grille** (lignes = techniciens, colonnes = jours de la semaine) et permet de **glisser-déposer** une intervention pour la réaffecter. Elle gère aussi les interventions sur **plusieurs jours**.

a) Affichage multi-jours. Une intervention apparaît sur **chaque** jour qu'elle couvre, grâce à un helper :

```
function interventionCoversDay(intervention, dayKey) {
  const startKey = toDateKey(new Date(intervention.dateDepart))
  const endKey   = intervention.dateRetour ? toDateKey(new Date(intervention.dateRetour)) : startKey
  return dayKey ≥ startKey && dayKey ≤ endKey   // le jour est-il entre début et fin ?
}
```

La carte affiche un badge « Jour 2/4 » et un style pointillé sur les jours de continuation.

b) Glisser-déposer (react-dnd). La carte est « draggable », la cellule est une « drop zone ». Au dépôt, on appelle `PUT /api/interventions/:id` en **conservant la durée** (on décale début ET fin du même nombre de jours) :

```
const durationDays = dayDiff(toDateKey(currentStart), toDateKey(currentEnd))
const newStart = mergeDateWithTime(targetDate, currentStart)
const newEnd   = new Date(newStart); newEnd.setDate(newEnd.getDate() + durationDays)

await apiFetch(`/api/interventions/${id}`, {
  method: 'PUT', token,
  body: { technicien_id: targetTechnicianId, date_depart: newStart.toISOString(), date_fin: newEnd.toISOString()
},
})
```

c) Filtrage par semaine. Des boutons « semaine précédente/suivante » changent un état `weekStart`, ce qui relance le chargement via `useEffect([weekStart, ...])`.

7.6 La validation — `ManagerValidation.tsx`

Deux onglets : « Rapports d'intervention » et « Notes de frais ». Pour les frais, le gestionnaire voit le **justificatif** et peut valider :

```
const handleValidateExpense = async (expenseId) => {
  await apiFetch(`/api/manager/expenses/${expenseId}/validate`, { method: 'PATCH', token })
  setExpenses((cur) => cur.filter((e) => e.id !== expenseId))   // disparaît de la liste "en attente"
}
```

Le lien vers le justificatif est construit à partir de `API_BASE_URL` + le chemin stocké en base :

```
<a href={`${API_BASE_URL}${expense.justificatifUrl}`} target="_blank" rel="noreferrer">
  Voir le justificatif
</a>
```

7.7 Les paiements — `ManagerPayments.tsx`

Liste les frais **validés** (`status=VALIDEE`), avec des cases à cocher. Le gestionnaire sélectionne plusieurs lignes, voit le total se calculer en direct, puis confirme le remboursement en lot :

```
const totalSelected = selectedIds.reduce((sum, id) =>
  sum + (expenses.find((e) => e.id === id)?.amount ?? 0), 0)

const handleConfirmPayment = async () => {
  await apiFetch('/api/manager/expenses/pay', {
    method: 'PATCH', token,
    body: { ids: selectedIds, reference: paymentData.reference || null },
  })
  await loadExpenses()    // les frais payés quittent la liste "à payer"
}
```

8. Trois scénarios de bout en bout

8.1 Créer un technicien

1. **UI** : « Ressources » → onglet Techniciens → « Ajouter » → formulaire.
2. **Front** : `apiFetch('/api/users', { method:'POST', token, body:{ nom, email, password, role:'TECHNICIEN' } })`.
3. **Back** : `requireRole(GESTIONNAIRE)` → hachage bcrypt → transaction `INSERT users` + `INSERT techniciens` → `201`.
4. **Front** : `loadResources()` recharge la liste, le nouveau technicien apparaît.

8.2 Planifier une intervention sur 3 jours

1. **UI** : « Planning » → clic sur une cellule → date de début + date de fin (3 jours plus tard).
2. **Front** : `apiFetch('/api/interventions', { method:'POST', body:{ … , date_depart, date_fin } })`.
3. **Back** : validation `date_fin ≥ date_depart` → `INSERT` avec `date_retour` → `201`.
4. **Front** : la carte s'affiche sur les 3 colonnes (badges « Jour 1/3 », « Jour 2/3 », « Jour 3/3 »).

8.3 Valider puis payer une note de frais

1. Le technicien crée une note de frais (mission + justificatif) → statut `EN_ATTENTE`.
2. **Validation** : le gestionnaire ouvre « Validation », voit le justificatif, clique « Valider » → `PATCH /validate` → statut `VALIDEE` + ligne d'audit.
3. **Paiement** : « Paiements » liste les `VALIDEE` ; il coche, confirme → `PATCH /pay` → statut `PAYEE` + audit.

9. Les patterns à réutiliser pour coder toi-même

9.1 Le squelette d'une route backend protégée

```
app.METHODE('/api/ressource/:id?', requireRole(ROLE_GESTIONNAIRE), async (req, res) => {
  try {
    // 1. Lire & valider les entrées (req.params, req.query, req.body)
    // 2. (option) Vérifier les droits/cohérence métier
    // 3. Exécuter la/les requête(s) SQL paramétrée(s) — transaction si plusieurs tables
    // 4. Répondre avec le bon code (200/201/ … ) et des données formatées
    return res.json({ success: true, /*  …  */ })
  } catch (error) {
    return res.status(500).json({ error: error.message })    // 5. Filet de sécurité
  }
})
```

9.2 Le squelette d'une page React (gestionnaire)

```
export default function MaPage() {
  const [items, setItems] = useState([])
  const [isLoading, setIsLoading] = useState(true)

  const load = async () => {
    const token = getManagerToken()
    try { setItems((await apiFetch('/api/ressource', { token })).items) }
    catch (e) { toast.error(e.message) }
    finally { setIsLoading(false) }
  }
  useEffect(() => { load() }, [])

  const save = async (payload, id?) => {
    if (id) await apiFetch(`/api/ressource/${id}`, { method:'PUT',  token, body: payload })
    else    await apiFetch('/api/ressource',       { method:'POST', token, body: payload })
    await load()    // recharger pour refléter l'état réel du serveur
  }

  return (
    <ManagerLayout>
      {isLoading ? <p>Chargement … </p> : /* tableau + modales */ null}
    </ManagerLayout>
  )
}
```

9.3 La check-list mentale

- [ ] **Sécurité d'abord** : toute route privée commence par `requireRole( … )`. Le frontend ne sécurise rien.
- [ ] **Requêtes paramétrées** (`$1` , `$2`) — jamais de concaténation de valeurs utilisateur.

- [ ] **Codes HTTP justes** : 200 (ok), 201 (créé), 400 (entrée invalide), 401 (non connecté), 403 (interdit), 404 (introuvable), 409 (conflit), 500 (erreur serveur).
- [ ] **Formate** les lignes SQL (`snake_case` → `camelCase`) avant de les renvoyer.
- [ ] **Recharge** la liste après une action (`load()`), plutôt que de bidouiller l'état local.
- [ ] `useEffect` pour charger au montage ; `useState` pour l'état ; `toast` pour les retours.
- [ ] **Redémarre** `npm run api` après toute modification de `server.js` .

Fin du document. Tu disposes maintenant d'une vue complète de l'espace gestionnaire : la base de données, chaque route API avec ses règles de sécurité, et chaque page React avec son cycle de vie. En suivant les patterns du §9, tu peux ajouter toi-même un nouveau module (par exemple un catalogue de « fournisseurs ») en quelques heures.